Linking AI agents in n8n to Orchestrate Project Reachy_Local_08.4.2

Note in project Reachy, I am using n8n in a self-hosted environment (local installation).
This community edition includes limited functionality. For example, example the system does not create numerous projects, only individual workflows.

Given that I'm using n8n to orchestrate an single project, I don't think this will be a problem. In this environment, what is the process to link numerous AI agents into a single system?

Answer the question in the context of project Reachy_Local_08.4.2.

Thought for 17s >

Short version: the "single system" isn't a special n8n feature, it's the fact that all 9 agents share **one n8n instance + one DB + one filesystem + one event schema**. You glue them together with webhooks, HTTP requests, and shared tables.

You already designed this for Reachy_Local_08.4.2 – we're just going to make the wiring explicit.

I'll break it into three pieces:

1. How agents are represented in n8n
2. How they're linked into one pipeline
3. What this looks like concretely for your 9 Reachy agents

---

1. One n8n instance, many agent workflows

In community/self-hosted n8n, "projects" are basically just **a collection of workflows**. That's fine for you, because Reachy_Local_08.4.2 *is* one project.

You treat each AI agent as **one n8n workflow**:

- `01_ingest_agent` → workflow handling ingest
- `02_labeling_agent` → workflow for labels/balance
- `03_promotion_agent` → workflow for temp→train/test
- … up through `09_observability_agent`

Each workflow gets:

- A **Trigger** node (usually a Webhook or Cron)
- A set of **processing nodes** (Postgres, HTTP, Code, SSH, etc.)
- One or more **"emit next event"** nodes (HTTP Request / Execute Workflow / Postgres insert)

So "linking agents" = deciding:

> *Which event should wake this workflow up, and what event does it emit when it's done?*

---

2. The glue: events + shared storage

You already defined the glue in AGENTS_08.4.2:

- Message envelope: every event has `schema_version`, `correlation_id`, `issued_at`, `source` etc.
- Shared DB: PostgreSQL 16 storing video metadata, splits, promotion logs, gates, etc.
- Shared FS: `/media/project_data/reachy_emotion/videos/{temp,train,test,thumbs,manifests}`

Practically, your linking surfaces are:

1. **Webhooks between agents**
   - Each agent exposes one or more **webhook URLs** (e.g. `/webhook/ingest.completed`, `/webhook/training.completed`).
   - Upstream agents call these via **HTTP Request nodes**.
2. **Shared Postgres tables**
   - Agents don't pass full video blobs around n8n.
   - They pass **IDs + paths**, and each workflow reads/writes rows in Postgres.
3. **Filesystem paths**

- Moves between `temp → train/test` happen on Ubuntu 1; agents just agree on the **canonical path format** and let FastAPI / shell scripts do the move.

So instead of “link projects,” you “link events and tables.”

---

3. Concrete process for Reachy_Local_08.4.2

You already have a full 9-agent layout described in `AGENTS_08.4.2.md` and implemented as JSON workflows under `n8n/workflows/`.

Here’s how they chain in your environment.

Step 0 — Import and name everything

1. In the n8n UI:
   - Import `01_ingest_agent.json` … `09_observability_agent.json`.
   - Name them with a clear prefix: `reachy_01_ingest`, `reachy_02_labeling`, etc.
   - Tag them with `reachy_local_08.4.2` so they’re visually grouped.
2. Configure shared credentials:
   - One **Postgres** credential for the Reachy DB.
   - One **SSH/HTTP** credential for calls to your FastAPI Media Mover, TAO host, Jetson, etc.

Now you have 9 separate “agents” living happily in a single n8n instance.

Step 1 — Define your event contract

You already have a “message envelope” spec in AGENTS: every message includes:

```
{
  "schema_version": "1.0",
  "correlation_id": "uuid-1234",
  "source": "web-ui",
  "event_type": "ingest.requested",
  "payload": { ... }
}
```

n8n-wise, that means:

- **Webhook Trigger node** parses this envelope.
- A **Function** node validates fields (`schema_version`, `correlation_id`, etc.).
- That `correlation_id` gets carried into all downstream HTTP calls and DB inserts.

This is the backbone that lets 9 workflows act like one “system” instead of 9 random scripts.

Step 2 — Chain the agents in order

2.1 Ingest → Labeling

- **Agent 1 (Ingest)**
  - Trigger: webhook `/webhook/ingest.requested` (UI or Luma generation calls this).
  - Nodes:
    - HTTP → Media Mover API to store video in `/videos/temp` and compute `sha256`.
    - Postgres → insert row into `media.video`.
    - HTTP → thumbnail generator.
  - Output: HTTP Request node calls **Agent 2** webhook with `event_type = "ingest.completed"` plus `video_id`, `sha256`, etc.
- **Agent 2 (Labeling)**
  - Trigger: webhook `/webhook/ingest.completed` or UI action.
  - Nodes:
    - Postgres → fetch/update video row, apply label (`happy` / `sad`) for **train** split only (test remains unlabeled).
    - WebSocket/HTTP → update UI with per-class counts + 50/50 status.
  - Output: when user hits “Promote,” the UI calls **Agent 3** webhook.

2.2 Labeling → Promotion → Reconciler

- **Agent 3 (Promotion / Curation)**
  - Trigger: `/webhook/promotion.requested`.
  - Nodes:
    - Postgres → check split constraints and 50/50 test ratio before moving.
    - HTTP/SSH → call FastAPI/OS script to move file `temp → train/test`.

    - Postgres → update `split` field and write promotion log.
  - Output:
    - On success, HTTP Request → **Agent 4** webhook (`event_type = "promotion.completed"`).
- **Agent 4 (Reconciler / Audit)**
  - Trigger: `/webhook/promotion.completed` **or** nightly Cron.
  - Nodes:
    - File + Postgres → compare checksums / detect orphans.
    - Function → rebuild `train_manifest.json` / `test_manifest.json`.
  - Output: HTTP → **Agent 5** when dataset is "ready_for_training = true".

2.3 Reconciler → Training → Evaluation → Deployment

- **Agent 5 (Training Orchestrator)**
  - Trigger: `/webhook/dataset.ready` or manual "Train Now" webhook.
  - Nodes:
    - SSH / HTTP → launch TAO EmotionNet train job with `emotion_train_2cls.yaml`.
    - HTTP → MLflow REST API to log run, metrics, dataset hash.
    - Polling or Webhook → check for job completion.
  - Output: on success, HTTP → **Agent 6** (`event_type = "training.completed"`, with artifact paths, F1, accuracy).
- **Agent 6 (Evaluation Agent)**
  - Trigger: `/webhook/training.completed`.
  - Nodes:
    - Postgres → verify test set balance and minimum samples per class.
    - SSH/HTTP → run evaluation script against validation split (not the unlabeled `/test`).
    - Function → compute confusion matrix, F1, latency metrics.
  - Output: if Gate B passes, HTTP → **Agent 7** (`event_type = "evaluation.passed"`).
- **Agent 7 (Deployment Agent)**
  - Trigger: `/webhook/evaluation.passed`.

- Nodes:
  - SSH → copy `.engine` to Jetson `/opt/reachy/models/emotion.engine`.
  - SSH → restart DeepStream service.
  - HTTP → call health/metrics endpoint on Jetson to verify FPS/latency thresholds.
- Output:
  - On success, HTTP → **Agent 9** (`deployment.completed`) to update dashboards.
  - On failure, roll back, then notify Observability.

2.4 Cross-cutting agents

- **Agent 8 (Privacy / Retention)**
  - Trigger: time-based (Cron) + manual webhook (`/webhook/privacy.purge`).
  - Nodes:
    - Postgres + File → purge items older than TTL in `/videos/temp`.
- **Agent 9 (Observability / Telemetry)**
  - Trigger: webhook from *any* other workflow (`obs.snapshot`) or periodic Cron.
  - Nodes:
    - Postgres/HTTP → collect metrics (queue depth, training duration, balance stats).
    - HTTP → push to Prometheus / Grafana.

Now all 9 workflows are **loosely coupled but strongly aligned**: they share the same DB, message format, and correlation IDs. That's your "single system."

---

4. What you actually do, step-by-step

If you want this as a concrete checklist:

1. **Import all 9 JSON workflows** into n8n and rename/tag them for Reachy.
2. **Create the shared Postgres credential** pointing at `reachy_local` DB on Ubuntu 1.
3. For each workflow:
   - Set **Webhook Trigger** URLs (internal-only, behind your LAN).
   - Wire nodes to:

    - Postgres (`media.video`, `promotion_log`, etc.)
    - FastAPI Media Mover
    - TAO/Jetson endpoints where needed.
4. In upstream workflows, add **HTTP Request nodes** that POST to the next agent's webhook with your standard event envelope.
5. Make sure every workflow **reads and writes** `correlation_id` so you can trace a single video through the entire loop.
6. Turn on **execution logging** in n8n so Observability Agent can scrape metrics.

That's it. You didn't need multiple "projects" in n8n; you needed **one well-structured set of workflows** that agree on how they talk to each other. You already did the hard thinking in `AGENTS_08.4.2.md` – n8n is just the wiring harness for it.

If I understand you correctly, output from one agentic workflow is linked to a separate agent via webhooks. Correct? For example, if agent 2 uses output from agent 1, agent 2 will include a webhook which is triggered by output from agent 1. Correct?

Exactly right.

Each agent's workflow ends by **sending an HTTP POST** (usually to another workflow's webhook URL) that contains its output payload.
That POST triggers the next agent's **Webhook Trigger** node, waking it up with that same payload.

So in your example:

- **Agent 1 (Ingest)** finishes processing → makes an HTTP Request to `http://localhost:5678/webhook/agent2_start` (or similar).
- **Agent 2 (Labeling)** has a **Webhook Trigger** node listening at `/webhook/agent2_start`.
- n8n receives the POST and immediately executes Agent 2's workflow with that JSON input.

It's basically a hand-off pattern:

> *"When I'm done, call your webhook with my output."*

This is how you create a chain of autonomous agents that act like one continuous system—even though each remains its own workflow.